

ANDROID DEVELOPMENT

B.E. (Electronics & Telecommunication / Electronics) | 2019 Pattern | Semester VIII | Elective V

END SEMESTER EXAM — 29 May 2025 | Max Marks: 70

Q1 a) Explain about Data Picker, Time Picker, ListView view, and Spinner View. [8 Marks]

i) Date Picker

DatePicker is a widget that allows the user to select a date — specifically the day, month, and year. It displays a calendar-style interface. You can use it directly in a layout or in a DatePickerDialog which is a popup window for date selection. The selected values are retrieved using `getDayOfMonth()`, `getMonth()`, and `getYear()` methods. Note that months are zero-indexed (0 = January, 11 = December).

ii) Time Picker

TimePicker allows the user to select a time — hours and minutes. It can be displayed in 12-hour (AM/PM) or 24-hour format. Like DatePicker, it can be used inline in a layout or inside a TimePickerDialog popup. The selected time is retrieved using `getHour()` and `getMinute()`. The TimePickerDialog provides a callback with the chosen hour and minute when the user confirms their selection.

iii) ListView

ListView is a ViewGroup that displays a list of scrollable items vertically. It is one of the most widely used widgets for showing lists of data such as contacts, messages, or products. ListView requires an Adapter to supply data and create the individual item views. The simplest adapter is ArrayAdapter which takes an array or list of strings and maps them to simple text views. You handle item selection through the OnItemClickListener which gives you the position of the tapped item.

iv) Spinner View

Spinner is a dropdown widget that lets the user select one item from a list. It is similar to a combo box or dropdown menu in desktop applications. When the user taps the spinner, a dropdown appears showing all available options. Like ListView, it uses an ArrayAdapter to populate the items. The `setDropDownViewResource()` method controls how the expanded dropdown list items appear. You handle selection through OnItemSelectedListener.

Q1 b) What is Android? Explain the versions and applications. [9 Marks]

Android is an open-source mobile operating system developed by Google, based on the Linux kernel. It is designed primarily for touchscreen mobile devices like smartphones and tablets, but is also widely used in smart TVs, smartwatches, cars (Android Auto), and IoT devices.

Android was originally developed by Android Inc. and was acquired by Google in 2005. The Open Handset Alliance (OHA), a group of companies including Google, HTC, and others, officially announced Android in 2007. The first commercial Android phone, the HTC Dream (T-Mobile G1), was released in 2008.

Key Features of Android

- Open source — The Android Open Source Project (AOSP) allows manufacturers and developers to freely use and modify the OS.
- Linux-based kernel — Provides memory management, process management, security, and hardware drivers.
- App development — Apps are written in Java or Kotlin and run on the Android Runtime (ART).
- Google Play Store — The official marketplace for downloading and distributing apps.

Android Versions

Version	API Level	Codename	Year
Android 2.3	9-10	Gingerbread	2010
Android 4.0	14	Ice Cream Sandwich	2011
Android 4.4	19	KitKat	2013
Android 5.0	21	Lollipop	2014
Android 6.0	23	Marshmallow	2015
Android 7.0	24	Nougat	2016
Android 8.0	26	Oreo	2017
Android 9	28	Pie	2018
Android 10	29	Q	2019
Android 11	30	R	2020
Android 12	31	S	2021
Android 13	33	Tiramisu	2022
Android 14	34	Upside Down Cake	2023

Android Applications

Category	Examples
Communication	WhatsApp, Gmail, Google Messages
Social Media	Instagram, Facebook, Twitter
Navigation	Google Maps, Waze
Entertainment	YouTube, Netflix, Spotify
Banking & Finance	HDFC, PhonePe, Google Pay
E-commerce	Amazon, Flipkart, Meesho
Education	Duolingo, Khan Academy, BYJU'S
Health & Fitness	HealthifyMe, Google Fit
Gaming	PUBG, Candy Crush, Clash of Clans

Q2 a) Explain the procedure/steps of Installing Android SDK Tools. [8 Marks]

The Android SDK (Software Development Kit) contains all the tools needed to build, test, and debug Android applications. The easiest way to install it is through Android Studio.

Step 1 — Check System Requirements

Before installation, ensure your computer meets the requirements: at least 8 GB RAM (16 GB recommended), at least 4 GB free disk space, a 64-bit operating system (Windows, macOS, or Linux), and Java JDK 8 or higher installed.

Step 2 — Download Android Studio

Visit the official Android developer website at developer.android.com/studio and download the Android Studio installer for your operating system.

Step 3 — Install Android Studio

Run the downloaded installer. During installation, choose the Standard setup option which automatically installs Android Studio, the Android SDK, the Android Emulator, and essential SDK tools like Platform-Tools and Build-Tools.

Step 4 — Configure SDK Manager

After installation, open Android Studio and go to Tools > SDK Manager. Under the SDK Platforms tab, select the Android versions you want to develop for and download them. Under the SDK Tools tab, ensure the following are installed: Android SDK Build-Tools, Android SDK Command-line Tools, Android Emulator, Android SDK Platform-Tools, and Intel HAXM (for faster emulator performance on Intel CPUs).

Step 5 — Set Environment Variables

To use Android tools from the command line, set the ANDROID_HOME environment variable to point to the SDK installation folder. Also add the tools/ and platform-tools/ sub-folders to the system PATH variable.

Step 6 — Create an Android Virtual Device (AVD)

Go to Tools > Device Manager in Android Studio and click Create Device. Choose a hardware profile (e.g., Pixel 6), select a system image (Android version), configure device settings, and click Finish. This creates a virtual Android device you can run on your computer for testing.

Step 7 — Create and Run a Project

Go to File > New > New Project, select a template, fill in the project name and package name, choose the minimum SDK version, and click Finish. To run the app, click the Run button (green triangle) and select either your AVD or a physical device connected via USB.

Q2 b) Briefly discuss about Linking Activities and Fragments. [9 Marks]

Linking Activities

Activities are linked to each other using Intents. An Intent is a message object that starts another activity and can carry data along with it. You create an Intent specifying the current context and the target Activity class. To pass data, use `intent.putExtra()` with a key-value pair. In the destination activity, retrieve the data using `getIntent()` and the appropriate getter method like `getStringExtra()` or `getIntExtra()`.

For cases where the first activity needs a result from the second activity (for example, picking a contact or an image), you use `startActivityForResult()` instead. The second activity sets a result using `setResult()` before finishing, and the first activity receives it in the `onActivityResult()` callback.

Linking Fragments to Activities

Fragments can be added to an activity statically (in the XML layout) or dynamically (in Java code at runtime). For dynamic fragment management, you use the `FragmentManager` obtained by calling

`getSupportFragmentManager()`. A `FragmentTransaction` is started by calling `beginTransaction()`, which provides methods to add, replace, or remove fragments.

Communication Between Fragment and Activity

- **Activity to Fragment** — Pass data using a `Bundle`. Create a `Bundle`, add key-value pairs to it, and call `fragment.setArguments(bundle)` before adding the fragment. Inside the fragment, retrieve the data with `getArguments().getString('key')`.
- **Fragment to Activity** — Define an interface inside the fragment. The activity implements this interface. When the fragment is attached to the activity (`onAttach` callback), it gets a reference to the interface. When something happens in the fragment, it calls the interface method, which the activity handles.
- **Fragment to Fragment** — The two fragments do not communicate directly. Instead, one fragment sends data to the activity via the interface approach, and the activity then passes it to the other fragment via arguments or a public method.

Q3) What is DVM? Explain the compiling and packaging process from the source file with diagram. [17 Marks]

The Dalvik Virtual Machine (DVM) is a register-based virtual machine that was specifically designed by Google to run Android applications efficiently on mobile devices with limited memory and processing power. Unlike the standard Java Virtual Machine (JVM) which is stack-based, DVM uses registers for operations, which makes it faster and more memory efficient on constrained hardware.

Key Characteristics of DVM

- DVM runs .dex (Dalvik Executable) files rather than standard Java .class files. The .dex format is a compact, optimized format designed for mobile use.
- Each Android application runs in its own separate DVM process, which provides isolation and security between apps.
- DVM was the primary runtime up to Android 4.4 (KitKat). From Android 5.0 (Lollipop) onward, it was replaced by ART (Android Runtime) which uses ahead-of-time (AOT) compilation for faster app execution.
- DVM is optimized for low memory, low CPU speed, and battery-constrained environments.

DVM vs JVM Comparison

Feature	DVM (Dalvik)	JVM (Java)
Architecture	Register-based	Stack-based
Bytecode format	.dex files	.class files
Designed for	Mobile devices	Desktop/Server systems
Memory usage	Low footprint	Higher memory use
Multiple instances	Yes — one per app	Typically one shared

Compiling and Packaging Process

The process of converting Java source code into an installable Android application (.apk file) goes through the following stages:

Stage	Tool Used	Input	Output
1. Java Compilation	javac (Java Compiler)	.java source files	.class files (Java bytecode)
2. DEX Conversion	dx or d8 tool	.class files	classes.dex (Dalvik bytecode)
3. Resource Processing	aapt (Asset Packaging Tool)	res/ folder + Manifest	resources.arsc + R.java
4. APK Packaging	APK Builder	.dex + resources + assets	Unsigned .apk file
5. Signing	apksigner / jarsigner	Unsigned .apk + keystore	Signed .apk file
6. Alignment	zipalign	Signed .apk	Optimized final .apk

Explanation of Each Stage

- Stage 1 — Java Compilation: The standard Java compiler (javac) reads your .java source files and converts them into .class files. These .class files contain standard Java bytecode, but they cannot run on Android directly.
- Stage 2 — DEX Conversion: The dx tool (or the newer and faster d8 tool in modern Android Studio) takes all the .class files and converts them into a single classes.dex file. This file contains Dalvik bytecode in a compact format optimized for mobile memory.

- Stage 3 — Resource Processing: The aapt tool processes the XML resource files in the res/ folder and compiles them into a binary format. It also generates the R.java file, which is an automatically created Java class that maps resource names to integer IDs used in code.
- Stage 4 — APK Packaging: The APK Builder combines all the compiled pieces: the classes.dex file, the compiled resources (resources.arsc), the binary AndroidManifest.xml, any native .so libraries, and the raw assets/ folder, into a single ZIP archive with a .apk extension.
- Stage 5 — Signing: Every APK must be digitally signed before it can be installed. Debug builds are signed automatically with a debug keystore. For release builds, you sign with your own private keystore using apksigner. The signature ensures the app's authenticity and integrity.
- Stage 6 — ZIP Alignment: The zipalign tool aligns all uncompressed data within the APK to 4-byte boundaries. This improves runtime memory mapping efficiency, making the app load faster and use less memory when running.

Q4 a) Explain the Components for Communication. [7 Marks]

Android provides several components and mechanisms that allow activities, services, and other parts of an application to communicate with each other, as well as communicate across different applications.

1. Intents

Intents are the primary communication mechanism in Android. An Intent is an object that carries a message describing what should be done. Explicit Intents name a specific target component (used within the same app), while Implicit Intents describe a general action (like opening a webpage) and let the system find an appropriate app to handle it. Intents can also carry extra data as key-value pairs.

2. Intent Filters

Intent Filters are declared in AndroidManifest.xml and tell Android which types of intents a particular component can respond to. When an implicit intent is sent, the system scans all apps' intent filters to find matching components and presents options to the user if there are multiple matches.

3. Broadcast and Broadcast Receivers

Broadcasts are system-wide messages sent by Android (for events like battery low or network change) or by apps (custom broadcasts). Broadcast Receivers are components that listen for and respond to these broadcasts. They don't have a UI but can start activities, show notifications, or trigger other actions.

4. Content Providers

Content Providers act as a bridge for sharing structured data between applications. They expose data through a URI-based interface. Android's built-in apps like Contacts and Media Store use Content Providers. Other apps access this data through ContentResolver using the standard CRUD operations.

5. Bound Services

A component can bind to a running service to interact with it directly — calling its public methods. This creates a direct client-server relationship between the activity (client) and the service (server). Communication happens through the IBinder interface returned by the service's onBind() method.

Q4 b) What is a Content Provider? How does it work? [10 Marks]

A Content Provider is one of the four fundamental Android application components. Its main purpose is to manage access to a structured set of data and allow that data to be shared between different applications in a secure and standardized way.

Why Content Providers Are Needed

Android apps run in sandboxed processes — they cannot directly access each other's files or databases. Content Providers solve this problem by acting as a controlled gateway. One app exposes its data through a Content Provider with specific permissions, and other apps access it through ContentResolver without ever touching the underlying database or files directly.

How a Content Provider Works

Data is accessed through a URI (Uniform Resource Identifier) that follows the format: content://authority/table/optional_id. The 'authority' is typically the package name of the provider app. The 'table' identifies which dataset to access. An optional ID at the end specifies a single row.

Core Operations

Operation	Method	Purpose
Query	query()	Retrieve data — returns a Cursor with results
Insert	insert()	Add a new record — returns the URI of new row

Update	update()	Modify existing records — returns rows affected
Delete	delete()	Remove records — returns number of rows deleted
Get Type	getType()	Returns MIME type of data at the given URI

Built-in Content Providers in Android

- **ContactsContract** — Provides access to the device's contacts database. You can read names, phone numbers, and email addresses.
- **MediaStore** — Provides access to media files including images, videos, and audio files stored on the device.
- **CalendarContract** — Provides access to the calendar and events database.
- **Telephony** — Provides access to SMS and MMS messages.

Access via ContentResolver

To read data from a Content Provider, you call `getContentResolver()` from your activity or context, and then use the `query()`, `insert()`, `update()`, or `delete()` methods with the appropriate URI. The system routes these calls to the correct Content Provider app automatically.

Q5 a) How was the communication establishing between a service and activity? [9 Marks]

There are several ways to establish two-way communication between an Activity and a Service in Android:

Method 1 — Bound Service (Most Common)

In this approach, the service creates an IBinder object through an inner class that extends Binder. The service returns this binder in its onBind() method. The activity binds to the service using bindService() and implements a ServiceConnection interface. When the connection is established, onServiceConnected() is called with the IBinder object. The activity casts this binder to the inner class type and calls getService() to get a direct reference to the service object. From this point, the activity can directly call any public method on the service, enabling full two-way communication.

The activity should unbind in onStop() using unbindService() to release the connection and allow the service to stop if nothing else is using it.

Method 2 — BroadcastReceiver

The service sends a broadcast using sendBroadcast() with an Intent containing result data. The activity registers a BroadcastReceiver in onResume() to listen for this specific broadcast action. When the broadcast arrives, onReceive() is called and the activity extracts the data from the intent. This is a simple and loosely coupled approach, suitable for one-way notifications from service to activity. The receiver should be unregistered in onPause() to avoid leaks.

Method 3 — Messenger (For IPC)

For inter-process communication (when the service runs in a different process), a Messenger is used. The service creates a Messenger backed by a Handler that processes incoming messages. The activity gets the service's Messenger from the IBinder and sends Message objects to it. The activity also passes its own Messenger as a reply-to field so the service can send messages back. This is more complex but works across process boundaries.

Comparison

Method	Best Used When	Complexity
Bound Service	Activity and service are in the same process	Medium
BroadcastReceiver	Simple one-way updates from service to activity	Low
Messenger / AIDL	Service runs in a separate process (IPC)	High

Q5 b) Explain the three animation systems used in Android applications. [9 Marks]

Android provides three distinct animation systems, each suited for different use cases:

1. View Animation (Tween Animation)

View Animation is the oldest and simplest animation system. It applies visual transformations to a View — such as moving it, rotating it, scaling it, or fading it — but importantly, it only changes the visual appearance and does not actually change the view's real position or properties.

Animations are typically defined in XML files stored in res/anim/. The main transformation types are: translate (moves the view), rotate (spins the view), scale (resizes the view), and alpha (changes transparency). Multiple transformations can be combined in an AnimationSet and played together or sequentially. The animation is applied by loading it with AnimationUtils.loadAnimation() and calling view.startAnimation().

The main limitation is that even though the view visually moves to a new location, its click area and actual position remain at the original location.

2. Property Animation (Animator)

Property Animation (introduced in API 11) is a more powerful and flexible system that actually changes the real property values of any object, not just its visual appearance. It overcomes the main limitation of View Animation.

The core class is `ObjectAnimator` which animates a specific property of an object (like `translationX`, `alpha`, `scaleX`, `rotation`) from one value to another over a specified duration. You can also use `AnimatorSet` to play multiple `ObjectAnimators` together (simultaneously) or in sequence. `ValueAnimator` is the base class that animates a value and lets you manually apply it to anything through a listener. This system works on any object, not just Views.

3. Drawable Animation (Frame Animation)

Drawable Animation works like a traditional animated GIF or flipbook. It plays a sequence of Drawable images (frames) one after another at specified intervals, creating the illusion of movement.

The animation is defined as an `AnimationDrawable` in an XML file in `res/drawable/`. Each item in the list specifies a drawable image and the duration to display it. You set this drawable as the background of an `ImageView`. You then cast the background to an `AnimationDrawable` and call `start()` to play it. You can set `oneshot=true` to play once or `false` to loop continuously.

Comparison

Feature	View Animation	Property Animation	Drawable Animation
Introduced in	API 1	API 11	API 1
Changes actual properties	No	Yes	No
Works on non-Views	No	Yes	No
Use case	Simple visual effects	Complex, real animations	Sprite/frame-by-frame animation

Q6 a) Define about binding activities to services. [9 Marks]

Binding an Activity to a Service means establishing a direct connection between the two so that the activity can call methods on the service and receive results. This is called a Bound Service. Unlike a started service that runs independently, a bound service lives only as long as at least one component is bound to it.

When to Use Binding

Binding is appropriate when you need ongoing interaction between the activity and service — for example, a music player where the activity needs to tell the service to play, pause, or skip tracks and receive the current track title in return. It is also used when multiple activities need to interact with the same service simultaneously.

The Binding Process

- Step 1 — The Service creates an inner class that extends Binder and provides a method like `getService()` that returns the service instance itself.
- Step 2 — The Service's `onBind()` method returns this Binder object to whoever binds to it.
- Step 3 — The Activity implements a `ServiceConnection` interface with two callback methods: `onServiceConnected()` (called when binding succeeds) and `onServiceDisconnected()` (called when the service unexpectedly disconnects).
- Step 4 — The Activity calls `bindService()` passing the Intent, the `ServiceConnection`, and the `BIND_AUTO_CREATE` flag (which creates the service if it is not already running).
- Step 5 — In `onServiceConnected()`, the `IBinder` is cast to the specific Binder subclass and `getService()` is called to get a direct reference to the service object.
- Step 6 — The Activity can now call any public method on the service directly.
- Step 7 — When the Activity no longer needs the service (typically in `onStop()`), it calls `unbindService()` to release the connection.

Lifecycle of Bound Service

Event	Callback Called
Activity calls <code>bindService()</code>	<code>Service.onCreate()</code> then <code>Service.onBind()</code>
Binding established	Activity's <code>ServiceConnection.onServiceConnected()</code>
Activity calls <code>unbindService()</code>	<code>Service.onUnbind()</code> then <code>Service.onDestroy()</code>
Service crashes	Activity's <code>ServiceConnection.onServiceDisconnected()</code>

Q6 b) Briefly discuss about deploying APK Files. [9 Marks]

Deploying an APK (Android Package) means distributing and installing the compiled Android application on devices. There are several methods:

1. Deploying via ADB for Testing

ADB (Android Debug Bridge) is a command-line tool included in the Android SDK Platform-Tools. During development and testing, you connect a physical device via USB (with USB Debugging enabled in Developer Options) or use an emulator and install the debug APK directly. This is fast and convenient for testing but the resulting APK cannot be published to the Play Store.

2. Generating a Signed Release APK

Before publishing an app, you must sign it with a private key. In Android Studio, go to Build > Generate Signed Bundle / APK. Choose APK (or the newer Android App Bundle format which is smaller). You need a keystore file — a file containing your private signing key. If you don't have one, create it by providing an alias, password, and certificate details. Choose the release build variant and click Finish. The signed APK is generated in the `app/release/` folder.

3. Signing and Alignment

Every APK must be signed to prove it came from you and hasn't been tampered with. For release builds, signing is done with your private keystore. After signing, the APK should be aligned using zipalign which optimizes the APK for runtime performance by aligning data on 4-byte boundaries.

4. Publishing to Google Play Store

To reach users through the official channel, create a developer account at play.google.com/console (one-time registration fee). Create a new app listing, upload the signed APK or AAB file, fill in the store listing (app description, screenshots, category), set the content rating, configure pricing, and submit for review. After Google's review process, the app goes live.

5. Direct APK Distribution (Sideloading)

You can distribute an APK directly by sharing the file through email, a website, or cloud storage. Users must enable the 'Install from Unknown Sources' option in their device settings to install apps outside the Play Store. This is useful for enterprise distribution, beta testing, or apps that don't qualify for the Play Store.

Method	Use Case	Play Store Distribution
ADB Install	Developer testing only	No
Direct APK share	Internal/beta testing, enterprise	No
Google Play Store	Public distribution	Yes

Q7 a) Write a short note on using SQLite databases for developing Android applications. [8 Marks]

SQLite is a lightweight, serverless, open-source relational database management system that is built directly into the Android operating system. Unlike traditional databases such as MySQL or Oracle that require a separate server process, SQLite stores the entire database as a single file on the device. This makes it ideal for mobile applications.

Key Characteristics

- No separate server process needed — the database engine is embedded within the app itself.
- The database file is stored at `/data/data/packageName/databases/` and is private to the app.
- Supports standard SQL syntax for creating tables, inserting, querying, updating, and deleting data.
- ACID compliant — ensures data integrity even if the app crashes or the device loses power.
- Suitable for storing structured data like user profiles, app settings, cached content, and local records.

SQLiteOpenHelper

The recommended way to work with SQLite in Android is to create a helper class that extends `SQLiteOpenHelper`. This class manages the database lifecycle. The `onCreate()` method is called once when the database is first created and is used to create the initial tables. The `onUpgrade()` method is called when the version number is increased, allowing you to add new tables or columns to the existing database without losing user data.

Operations

All database operations use an `SQLiteDatabase` object obtained from the helper. To write data, call `getWritableDatabase()`. To only read data, call `getReadableDatabase()` which is more efficient. Data is inserted and updated using `ContentValues` objects that store column-value pairs. Data is read by executing queries that return a `Cursor`, which you iterate through to extract results. Always close the `Cursor` and database objects after use to prevent memory leaks.

When to Use SQLite vs Other Storage

Use SQLite When	Use SharedPreferences When	Use Files When
Storing structured, relational data	Storing simple key-value settings	Storing raw files, images, documents
Data has multiple fields and relationships	A few settings or user preferences	Large binary data
Need to query and filter data	Fast retrieval of simple values	Audio, video, downloaded content

Q7 b) Explain JSON Parsing and JSON Web Service. [10 Marks]

What is JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based format for storing and transmitting structured data. It is easy for humans to read and write, and easy for machines to parse and generate. JSON is the most widely used format for exchanging data between a mobile app and a web server.

JSON data is organized as key-value pairs within curly braces for objects, and square brackets for arrays. Keys must be strings. Values can be strings, numbers, booleans, null, arrays, or other objects.

JSON Parsing in Android

Android provides the `org.json` package for parsing JSON. The two main classes are `JSONObject` (for JSON objects enclosed in curly braces) and `JSONArray` (for JSON arrays enclosed in square brackets).

To parse a JSON object, create a new `JSONObject` from the JSON string. Use `getString('key')` to read a string value, `getInt('key')` for an integer, `getBoolean('key')` for a boolean, and `getJSONObject('key')` or `getJSONArray('key')` for nested objects or arrays. For arrays, use a for loop with the array's `length()` method to iterate through each element.

JSON parsing can throw a `JSONException` if the data is malformed or a key doesn't exist, so it should always be wrapped in a try-catch block.

JSON Web Services

A JSON Web Service is a REST API endpoint on a web server that returns data in JSON format in response to HTTP requests. Mobile apps fetch this data over the internet and display it to users.

The basic flow is: the app makes an HTTP GET request to the API URL, the server responds with a JSON string, the app parses the JSON using `JSONObject` or `JSONArray`, and the extracted data is displayed in the UI.

Important rules: network requests must always be done on a background thread (never on the main UI thread) because they can be slow and would freeze the app. Android enforces this and throws a `NetworkOnMainThreadException` if you try. The `INTERNET` permission must be declared in `AndroidManifest.xml`.

Popular libraries like Volley and Retrofit simplify web service calls significantly. Volley handles background threading, caching, and error handling automatically. You simply create a `JsonObjectRequest` with the URL and implement success and error callbacks, and Volley handles all the networking details.

Q8 a) Explain widgets, its types and list them. [6 Marks]

In Android, the term 'widget' is used in two different contexts:

1. UI Widgets (Views used inside app layouts)

These are the basic UI building blocks used to construct the interface of an app screen. Every visible element in an Android layout is a widget (a View or a subclass of View).

Category	Widget Names
Text Input	EditText, AutoCompleteTextView, MultiAutoCompleteTextView
Selection	CheckBox, RadioButton, RadioGroup, Switch, ToggleButton
Display	TextView, ImageView, VideoView, ProgressBar, RatingBar
Buttons	Button, ImageButton, FloatingActionButton
Lists/Grids	ListView, GridView, RecyclerView, Spinner
Date & Time	DatePicker, TimePicker, Chronometer, AnalogClock
Sliders	SeekBar, RangeSlider
Containers	ScrollView, HorizontalScrollView, ViewPager, CardView
Notifications	Toast, Snackbar, AlertDialog, ProgressDialog

2. App Widgets (Home Screen Widgets)

App Widgets are miniature app views that can be placed directly on the Android home screen. They provide at-a-glance information without requiring the user to open the app. Examples include a weather widget showing current temperature, a music player widget with play/pause controls, a clock widget, or a calendar widget showing upcoming events.

App Widgets are created using AppWidgetProvider (a special BroadcastReceiver), a RemoteViews layout for the widget's appearance, and an AppWidgetProviderInfo XML file that specifies the widget's size, update frequency, and initial layout. They are registered in AndroidManifest.xml as a receiver.

Q8 b) What is the meaning of location-based services? Explain the methods of Location Objects. [12 Marks]

Location-Based Services (LBS)

Location-Based Services are applications and services that use the geographic position of a mobile device to provide contextually relevant information, functionality, or services. The device's location is determined using GPS satellites, Wi-Fi networks, or cellular towers.

Examples of Location-Based Services

- Navigation apps like Google Maps that provide turn-by-turn directions based on where you are.
- Ride-hailing apps like Uber or Ola that find drivers near your location.
- Weather apps that automatically show weather for your current city.
- Nearby search — finding restaurants, ATMs, hospitals, or petrol stations around you.
- Geo-fencing — sending notifications when you enter or leave a specific geographic area.
- Asset tracking — monitoring the location of vehicles, deliveries, or equipment.
- Augmented Reality apps that overlay information on real-world locations.

The Location Object

When Android detects the device's position, it creates a Location object (android.location.Location) that holds all the relevant location information. This object is passed to your app through the LocationListener's onLocationChanged() callback.

Key Methods of the Location Object

Method	Return Type	Description
getLatitude()	double	Returns the latitude in decimal degrees (positive = North, negative = South)
getLongitude()	double	Returns the longitude in decimal degrees (positive = East, negative = West)
getAltitude()	double	Returns the altitude in meters above the WGS 84 reference ellipsoid (if available)
getSpeed()	float	Returns the speed in meters per second (if available)
getAccuracy()	float	Returns the estimated horizontal accuracy in meters — smaller is more precise
getBearing()	float	Returns the direction of travel in degrees clockwise from North (0 to 360)
getTime()	long	Returns the UTC timestamp of when the location was determined (in milliseconds)
getProvider()	String	Returns the name of the provider that generated the location (gps or network)
distanceTo(Location)	float	Returns the distance in meters between this location and another Location object
hasAltitude()	boolean	Returns true if altitude information is available
hasSpeed()	boolean	Returns true if speed information is available
hasAccuracy()	boolean	Returns true if accuracy information is available

Location Providers

Provider	How It Works	Accuracy	Battery Use
GPS_PROVIDER	Uses signals from GPS satellites	Very high (meters)	High
NETWORK_PROVIDER	Uses Wi-Fi and cell tower signals	Medium (100m+)	Low
PASSIVE_PROVIDER	Reuses location found by other apps	Varies	Very Low